Sleep Stage Classification from EEG Spectrograms

1. Nguyen The Khai 23BI14205
2. Ngo Minh Phuoc 23BI14361
3. Tran Gia Khanh 23BI14218
4. Ngo Xuan Kien 23BI14239
5. Tran Ngoc Hung 23BI14181
6. Nguyen Thanh Vinh 23BI14453

***Abstract*—This paper presents a structured dataset of time–frequency spectrogram images derived from single-channel electroencephalography (EEG) recordings for automated sleep stage classification. The data are segmented into 30-second epochs following the American Academy of Sleep Medicine (AASM) guidelines and organized into five standard sleep stages: Wake (W), N1, N2, N3, and REM. Spectral representations are generated using the Short-Time Fourier Transform (STFT) with fixed preprocessing parameters to ensure consistent physical magnitude interpretation across subjects. The frequency range is restricted to 0.5–30 Hz, with 0.5 Hz resolution, and power values are converted to decibel (dB) scale and normalized.**

I. Introduction

Spectrogram of the brain plays crucial role for brain status information, it provides information for the doctor to provide diagnosis or monitoring. Brain signal is often represented by EEG signal, in our dataset we used for the project, single channel EEG signal was used for inform sleep signal, but not in signal form like radio signal, 2D signal.

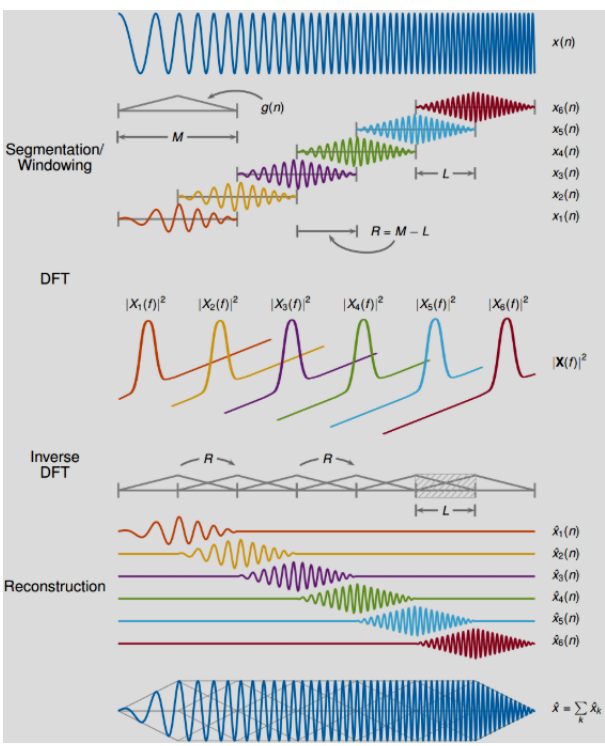


Fig. 1: This illustrates algorithm to convert an audio signal to a spectrographic representation by applying the FFT on windowed portions and summing these values together

A convolutional neural network (CNN) is a type of deep learning model designed to process grid-like data, such as images. It uses convolutional layers to automatically detect spatial patterns by applying filters across local regions of the input. So CNN may not suitable for signal processing task. To solve the problem, we change the representation form from signal form to image form.

Signal Data

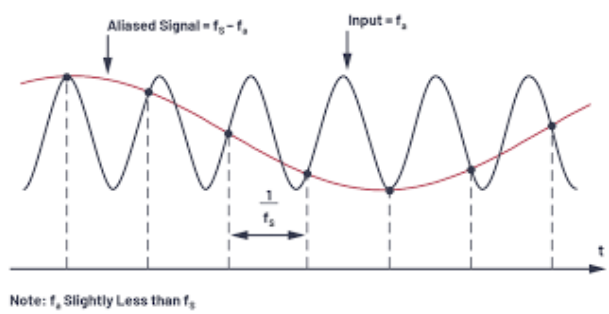


Fig. 2: Signal data form

- **Amplitude**: This is the height of the sound wave, indicating the intensity of the sound. A larger amplitude corresponds to a louder sound.

- **Period**: This is the time it takes for a signal to complete one full wave cycle.

- **Frequency**: This is the number of wave cycles completed in one second. Frequency is measured in Hertz (Hz) and is the inverse of the period.For example, if a wave completes one cycle in 1/100th of a second, its frequency would be 100 Hz.

Image data

Image data is represented as a structured numerical tensor encoding pixel intensity values arranged in a spatial grid. An image is modeled as a 3D tensor:

$$\mathbf{X} \in R^{H \times W \times C}$$

where:

- H = Height (number of rows of pixels)
- W = Width (number of columns of pixels)
- C = Number of channels

Typical cases:

- Grayscale image: $C = 1$
- RGB image: $C = 3$ (Red, Green, Blue channels)

For a dataset of N images, the full input tensor becomes:

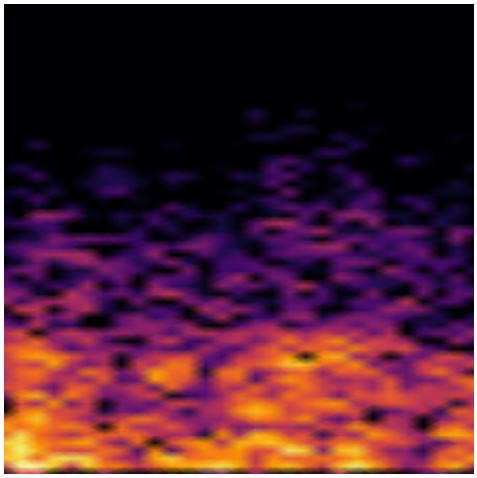


Fig. 3: Image data form

$$\mathbf{X} \in R^{N \times H \times W \times C}$$

(Or $N \times C \times H \times W$ depending on the framework e.g., PyTorch uses channel-first format.)

II. Dataset Structure - Overview

This dataset consists of a collection of time-frequency spectrogram images generated from single-channel EEG signals. The data is segmented into 30-second epochs, following the AASM (American Academy of Sleep Medicine) standard for sleep stage classification. It is specifically designed for training Deep Learning models (CNNs) and exploring Transfer Learning in the field of Digital Health. The dataset follows a "folder-per-class" structure, making it natively compatible with standard data loaders like `Keras ImageDataGenerator` or `PyTorch ImageFolder`.

- **W:** Wakefulness
- **N1:** Light Sleep (Transition)
- **N2:** Light Sleep (Spindles/K-complexes)
- **N3:** Deep Sleep (Slow Wave Sleep)
- **REM:** Rapid Eye Movement

File Naming Convention

Each image preserves patient traceability and temporal order to prevent **data leakage** during train/test splits.

- **Format:** `Sub{ID}_ep{Index}.png`
- **Sub{ID}:** Patient identifier (e.g., `Sub001`)
- **ep{Index}:** Sequential index of the 30s epoch within the original recording

Technical Specifications

- **Dimensions:** 300 $\times$ 300 pixels
- **Format:** PNG (Lossless compression)
- **Channels:** 3 (RGB)
  Note: While the data is scalar (power), it is saved in RGB to leverage pre-trained models.
- **Visuals:** Raw data only. No axes, labels, or margins are included.

Signal Preprocessing & Spectral Mapping

To ensure color intensity represents the same physical magnitude across all patients, fixed parameters were applied:

- **Transform:** Short-Time Fourier Transform (STFT)
- **Window:** 2 seconds (1s overlap)
- **Frequency Range:** 0.5 Hz – 30 Hz (Physiological ROI)
- **Resolution:** 0.5 Hz, allowing clear distinction in slow-wave activity
- **Color Map:** *Inferno* (Perceptually uniform, high contrast)
- **Scaling (Z-Normalization)**
- **Power:** Converted to Decibels (dB)
- **Clipping:** Lower limit (Black) at -5 dB; Upper limit (Yellow) at $+25$ dB

Original Source & Attribution

This dataset is a processed derivative of the DODH (Database of Digital Health).

- **Original Data:** https://github.com/Dreem-Organization/dreem-learning-open
- **Processing by:** Daniela Gluj

III. Sleep Stage Cycle Exploration

This section will inform you knowledge about human sleep cycle. When thinking about getting the sleep you need, it's normal to focus on solely how many hours of sleep you get. While sleep duration is undoubtedly important, it's not the only part of the equation. .

There are four sleep stages in Figure 4 , including one for rapid eye movement (REM) sleep and three that form non-REM (NREM) sleep, and each plays a part in allowing you to wake up refreshed. Understanding the sleep cycle also helps explain how certain sleep disorders, including insomnia and obstructive sleep apnea, can impact a person's sleep and health.

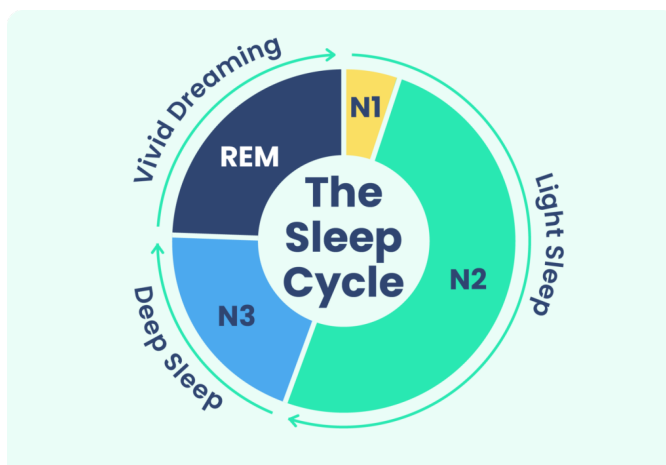
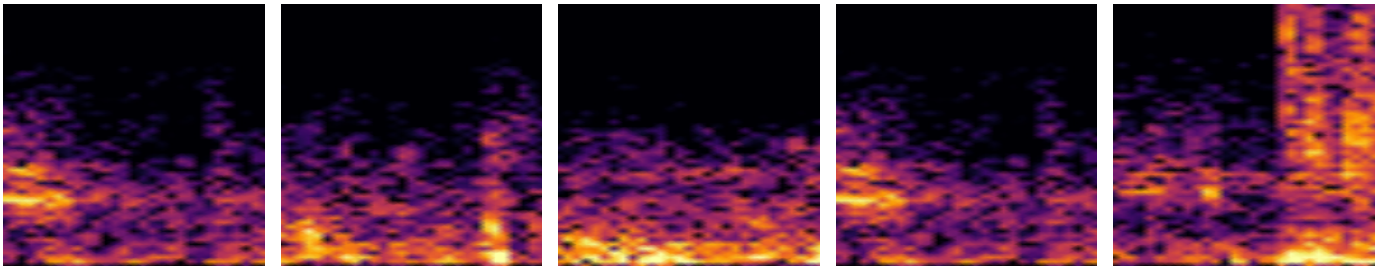


Fig. 4: Sleep cycle principle

Sleep isn't uniform. Over the course of the night, you go through several rounds of the sleep cycle, which is composed of four stages. In a typical night, a person goes through four to six sleep



(a) (Stage N1) (b) (Stage N2) (c) (Stage N3) (d) (Stage REM) (e) (Fully wake)

Fig. 5: Five stage of brain signal in sleeping cycle

A. *Stage N1*

Stage 1, also called N1, occurs when a person first falls asleep and usually lasts one to seven minutes. During N1 sleep, the body hasn't fully relaxed, though the body and brain activities start to slow with periods of brief movements. There are light changes in brain activity associated with falling asleep in this stage.

B. *Stage N2*

During stage 2, or N2, the body enters a more subdued state where body temperature drops, muscles relax, and heart rate and breathing slow. At the same time, eye movement stops and brain activity slows, though there are short bursts of activity that help your resist being woken up by external stimuli.

Stage 2 sleep can last for 10 to 25 minutes during the first sleep cycle, and each N2 stage can become longer during the night

C. *Stage N3*

Stage 3 sleep is also known as N3 or deep sleep, and it's harder to wake someone up if they're in this phase. Muscle tone, pulse, and breathing rate decrease in N3 sleep, and brain activity has an identifiable pattern of what are known as delta waves. For this reason, stage 3 may also be called delta sleep or slow-wave sleep (SWS).

Experts believe that this stage is critical to restorative sleep, allowing for bodily recovery and growth. It may also bolster the immune system and other key bodily processes. Even though brain activity is reduced, there's evidence that deep sleep contributes to insightful thinking, creativity, and memory. This stage take most of time during the sleep in general, approximately half of your sleeping session.

D. *REM Stage*

During stage 4, also called REM sleep, brain activity picks up, nearing levels seen when you're awake. At the same time, the body experiences atonia, which is a temporary paralysis of the muscles, with two exceptions: the eyes and the muscles that control breathing. Even though they're closed, the eyes move rapidly, which is how this stage gets its name.

REM sleep is believed to be essential to cognitive functions like memory, learning and creativity. REM sleep is known for the most vivid dreams, which is explained by the significant uptick in brain activity. Dreams can occur in any sleep stage, but they're less common and intense in the NREM periods *(N1, N2, N3)*.

Source: Stages of Sleep: What Happens in a Normal Sleep Cycle?

IV. Data Exploratory and Preprocessing

The dataset contains images of spectrogram of brain signal from five class N1, N2, N3, REM, W (Wake), spectrograms inform our brain signal emission during different stage of sleeping. Spectrograms inform two main information are **Time(x)** and **Frequency(y)**,we witness more in Figure 5. Each types of brain signal correspond to different brain states.

Frequency Band	Range	Meaning
Delta	0.5–4 Hz	Deep sleep
Theta	4–8 Hz	Drowsiness, memory processing
Alpha	8–13 Hz	Relaxed, eyes closed
Beta	13–30 Hz	Active thinking, concentration
Gamma	30–100 Hz	High-level cognition, perception

TABLE I: EEG Frequency Bands and Their Meanings

This is the class distribution in the dataset, we can see that there is a big gap between each class to the others, especially class N2, which can cause bias and poor prediction performance.

class	num_images	num_patients
W	1008	5
N1	317	5
N2	2047	5
N3	523	5
REM	754	5

TABLE II: Dataset distribution by sleep stage

Classes Samples distributions

Even the dataset has a bias distribution of each class, there is noway for us to handle this problem by generate more images data because of the limitation of the specified tool for this purpose. Not only images data generating can cause overfit,b ut also provide hallucination to the model. In the detail, even single false detail can cause false predictions. The solution for this problem is use Rarity Weight shifting. This method treats class with fewer samples more important (higher weight) than

Fig. 6: Class Distribution

a class with more sample, for example: Class N2 Figure 6.By applying class weighting, misclassification of minority classes incurs a larger penalty in the loss function, encouraging the model to learn more balanced decision boundaries across all classes.

Data Images Augmentation

Augmentation is applied to spectrogram inputs to improve model robustness. The frequency mask size of 8 Hz means that up to eight frequency bins can be randomly masked, while the time mask size of 5 steps removes short temporal segments from the spectrogram. Two masks are applied in both the frequency and time dimensions, increasing variability in the training data. The augmentation is applied with probability $p = 0.5$, meaning that only half of the training samples are augmented, which helps balance data diversity while preserving original samples.

V. Model Architecture

This project evaluates four classification models on EEG spectrogram images: a custom-designed CNN trained from scratch, and three standard architectures from ImageNet pretrained weights including ResNet-18, EfficientNet-B0, and MobileNet V2. All models receive a $300 \times 300 \times 3$ RGB spectrogram image as input and produce a 5-class output (W, N1, N2, N3, REM). For pretrained models, a two-phase training strategy was applied: the backbone is frozen for the first few epochs while only the classification head is trained, then the entire network is fine-tuned with a lower learning rate for the backbone ($\text{LR}_{\text{backbone}} = 10^{-5}$) and a higher rate for the head ($\text{LR}_{\text{head}} = 10^{-3}$).

A. *Custom CNN*

The Custom CNN is designed from scratch without any pretrained weights. The feature extractor consists of five convolutional blocks with progressively increasing channel depths: 32, 64, 128, 256, and 512 filters, respectively. Each block follows the pattern Conv2d $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Pooling.

Block	In→Out ch	Kernel	Norm	Act.	Pooling
1	$3 \rightarrow 32$	3×3	BN	ReLU	MaxPool 2×2
2	$32 \rightarrow 64$	3×3	BN	ReLU	MaxPool 2×2
3	$64 \rightarrow 128$	3×3	BN	ReLU	MaxPool 2×2
4	$128 \rightarrow 256$	3×3	BN	ReLU	MaxPool 2×2
5	$256 \rightarrow 512$	3×3	BN	ReLU	AdaptiveAvgPool 1×1

TABLE III: Custom CNN feature extractor blocks. All convolutions use padding = 1. BN = Batch Normalization.

MaxPool2d with stride 2 is used in the first four blocks to reduce spatial dimensions at each stage, reducing a 300×300 input down to 18×18 before Block 5. The final block uses Adaptive Average Pooling to collapse any remaining spatial dimensions to a 1×1 feature map, producing a 512-dimensional vector regardless of input resolution. The classification head takes the flattened 512-dimensional feature vector and passes it through two fully connected layers with dropout.

Property	Value
Input size	$300 \times 300 \times 3$
Conv blocks	5
Feature channels	32 / 64 / 128 / 256 / 512
Classifier	FC(512→256) + FC(256→5)
Pretrained	No (trained from scratch)

TABLE IV: Custom CNN architecture summary

B. *ResNet-18*

ResNet-18 [4] was introduced by Kaiming He et al. (2015) to address the vanishing gradient problem in deep networks. Its defining contribution is the residual (skip) connection, instead of learning $\mathcal{H}(x)$ directly, each block learns the residual $\mathcal{F}(x) = \mathcal{H}(x) - x$, so the block output becomes $\mathcal{F}(x) + x$.

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$$

This identity shortcut allows gradients to flow directly through the skip path, enabling reliable training of much deeper networks. ResNet-18 is the smallest member of the ResNet family, built with 18 weight layers organized as a Fig. 7. The original ImageNet fully connected layer (FC $512 \rightarrow 1000$) is replaced with a new FC layer ($512 \rightarrow 5$) for 5-class sleep stage classification.

Property	Value
Input size	$300 \times 300 \times 3$
Residual blocks	8 BasicBlocks (4 groups $\times$ 2)
Feature depth	$64 \rightarrow 128 \rightarrow 256 \rightarrow 512$
Classifier head	FC($512 \rightarrow 5$)
Pretrained	Yes (ImageNet)
Total parameters	≈11.2 M

C. *EfficientNet-B0*

EfficientNet [5] was introduced by Tan and Le (2019) from Google Research. Its key innovation is compound scaling, instead of scaling only depth, width, or resolution independently,

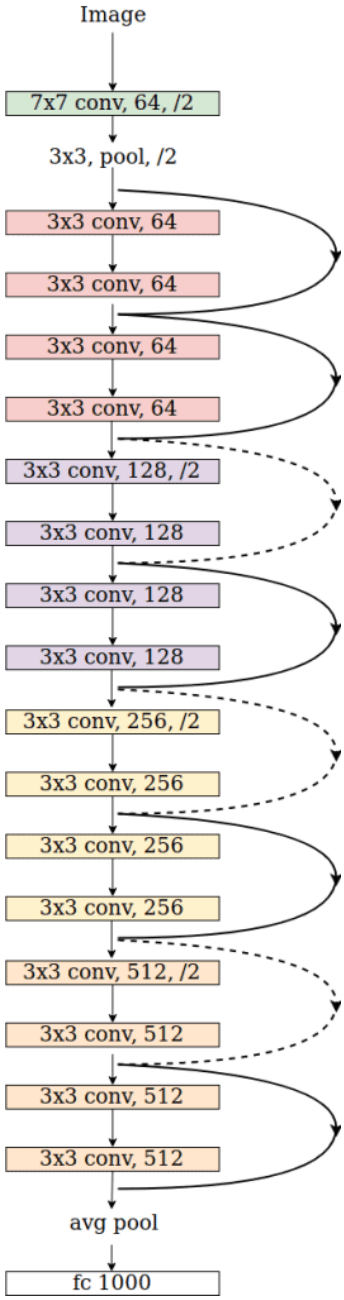


Fig. 7: Resnet 18 Architecture

all three dimensions are scaled jointly using a single compound coefficient ϕ:

$$\text{depth: } d = \alpha^\phi, \quad \text{width: } w = \beta^\phi, \quad \text{resolution: } r = \gamma^\phi$$

$$\text{subject to: } \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2, \quad \alpha \geq 1,\ \beta \geq 1,\ \gamma \geq 1$$

EfficientNet-B0 ($\phi = 0$) is the baseline network, consisting of a stem convolution followed by 7 Mobile Inverted Bottleneck Convolution (MBConv) stages and a final classification head. Each MBConv block expands the channel dimension by a factor of 6 using a $1{\times}1$ pointwise convolution, applies a $k{\times}k$ depthwise convolution, then projects back down via another $1{\times}1$ convolution — a highly parameter efficient design. Additionally, each MBConv incorporates a Squeeze-and-Excitation (SE) module: global average pooling $\rightarrow$ FC $\rightarrow$ ReLU $\rightarrow$ FC $\rightarrow$ Sigmoid, which recalibrates channel-wise feature responses.

Stage	Operator	Resolution	Channels	Layers
1	Conv3×3	300×300	32	1
2	MBConv1, k3×3	150×150	16	1
3	MBConv6, k3×3	150×150	24	2
4	MBConv6, k5×5	75×75	40	2
5	MBConv6, k3×3	38×38	80	3
6	MBConv6, k5×5	19×19	112	3
7	MBConv6, k5×5	19×19	192	4
8	MBConv6, k3×3	10×10	320	1
9	Conv1×1 & Pooling & FC	10×10	1280	1

TABLE V: EfficientNet-B0 baseline network. Each row describes a stage i with $\hat{L}_i$ layers, input resolution $\langle \hat{H}_i, \hat{W}_i \rangle$, and output channels $\hat{C}_i$.

The pretrained FC layer (1280→1000) is replaced with FC(1280→5). Because of compound scaling, EfficientNet-B0 achieves competitive accuracy while remaining compact.

Property	Value
Input size	$300 \times 300 \times 3$
MBConv stages	7 stages, 16 total blocks
Final feature dim	1280
Classifier head	FC(1280 $\rightarrow$ 5)
Pretrained	ImageNet
Total parameters	≈5.3 M

TABLE VI: EfficientNet-B0 architecture summary

D. *MobileNet V2*

MobileNet V2 [6] was developed by Sandler et al. (2018) at Google and is designed for deployment on resource-constrained devices. Its core building block is the Inverted Residual with Linear Bottleneck: unlike the traditional bottleneck that compresses $\rightarrow$ processes $\rightarrow$ expands, MBConv first expands the feature channels, applies a depthwise convolution, then compresses back to a low-dimensional representation.

The inverted residual block is formally:

$$\mathbf{y} = \text{PW-project}(\text{DW-conv}(\text{PW-expand}(\mathbf{x}))) + \mathbf{x}$$

where PW-expand uses a $1{\times}1$ convolution with expansion factor $t = 6$, DW-conv is a $3{\times}3$ depthwise convolution, and PW-project is a $1{\times}1$ projection (no activation). The linear bottleneck at the end prevents information loss that would otherwise occur if a non-linearity were applied to the low-dimensional output. Skip connections are used only when input and output dimensions match.

Input	Operator	Output
$h \times w \times k$	1×1 conv2d, ReLU6	$h \times w \times (tk)$
$h \times w \times tk$	3×3 dwise $s = s$, ReLU6	$\frac{h}{s} \times \frac{w}{s} \times (tk)$
$\frac{h}{s} \times \frac{w}{s} \times tk$	linear 1×1 conv2d	$\frac{h}{s} \times \frac{w}{s} \times k'$

TABLE VII: Bottleneck residual block transforming from k to k' channels, with stride s, and expansion factor t.

Input	Operator	t	c	n	s
$300^2 \times 3$	conv2d	–	32	1	2
$150^2 \times 32$	bottleneck	1	16	1	1
$150^2 \times 16$	bottleneck	6	24	2	2
$75^2 \times 24$	bottleneck	6	32	3	2
$38^2 \times 32$	bottleneck	6	64	4	2
$19^2 \times 64$	bottleneck	6	96	3	1
$19^2 \times 96$	bottleneck	6	160	3	2
$10^2 \times 160$	bottleneck	6	320	1	1
$10^2 \times 320$	conv2d 1x1	–	1280	1	1
$10^2 \times 1280$	avgpool 10x10	–	–	1	–

TABLE VIII: MobileNet V2 architecture. Each line describes a sequence of 1 or more identical (modulo stride) layers, repeated n times. c = output channels, t = expansion factor, s = stride of first layer. All spatial convolutions use 3×3 kernels. The expansion factor t is always applied to the input size as described in Table VII

.

The use of ReLU6 rather than standard ReLU improves robustness in fixed-point arithmetic. The 1×1 linear projection in each block avoids the information loss that ReLU causes on low-dimensional manifolds[6]. This design achieves high efficiency: MobileNet V2 has only $\sim$3.4M parameters yet maintains competitive accuracy, making it attractive when computational resources are limited.

Property	Value
Input size	$300 \times 300 \times 3$
Inverted residual blocks	17 blocks across 7 layers
Expansion factor	$t = 6$ (except Layer 1: $t = 1$)
Final feature dim	1280
Classifier head	FC($1280 \to 5$)
Pretrained	ImageNet
Total parameters	$\approx$3.4 M

TABLE IX: MobileNet V2 architecture summary

VI. Results

This section reports the test-set performance of all four models evaluated on Subject 015 (the held-out test subject). The test set contains 929 spectrogram epochs distributed across the five sleep stages: W (57), N1 (58), N2 (409), N3 (144), REM (261).

A. *Evaluation Metric: Cohen's Kappa*

While overall accuracy is the most intuitive metric, it is misleading in the presence of class imbalance. In our test set, N2 accounts for 409 out of 929 samples (44%). A classifier that predicts N2 for every input would therefore achieve 44% accuracy without learning anything useful. Standard accuracy treats all samples equally regardless of class frequency, which inflates the score for majority-class-dominated datasets.

Cohen's Kappa (κ) corrects for this by measuring agreement between the model's predictions and the ground truth beyond what would be expected by chance. It is defined as:

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

where p_o is the observed agreement (overall accuracy) and p_e is the expected agreement by chance, computed from the marginal distributions of predicted and true labels:

$$p_e = \sum_c \frac{n_{c,\text{true}} \times n_{c,\text{pred}}}{N^2}$$

Here $n_{c,\text{true}}$ is the number of true instances of class c, $n_{c,\text{pred}}$ is the number of predicted instances of class c, and N is the total number of samples. The range of κ is $(-\infty, 1]$: a value of 1 indicates perfect agreement, 0 means the model performs no better than random chance, and negative values indicate performance worse than chance[8].

The standard interpretation scale for κ in clinical applications is:

κ range	Interpretation
< 0	Less than chance
0.0–0.20	Slight agreement
0.21–0.40	Fair agreement
0.41–0.60	Moderate agreement
0.61–0.80	Substantial agreement
0.81–1.00	Almost perfect agreement

TABLE X: Cohen's Kappa interpretation scale (Landis & Koch, 1977)

Cohen's Kappa is particularly important in sleep staging because: (1) the class distribution is naturally skewed, as N2 dominates a typical night of sleep; (2) scorers are validated against human expert agreement, and κ is the standard measure used in clinical PSG inter-rater reliability studies.

B. *Overall Model Comparison*

Table XI summarises the overall test accuracy and Cohen's Kappa score for each model. ResNet-18 achieves the highest test accuracy (86.33%) and Kappa (0.8086), followed closely by MobileNet V2 (86.11%, $\kappa = 0.8027$). Custom CNN and EfficientNet-B0 score slightly lower. All four models exceed 84% accuracy, and all Kappa scores fall in the "strong agreement" range ($\kappa > 0.77$).

Model	Test Acc (%)	Macro F1	Kappa
Custom CNN	85.04	0.7875	0.7881
ResNet-18	**86.33**	**0.8092**	**0.8086**
EfficientNet-B0	84.39	0.7390	0.7783
MobileNet V2	86.11	0.7999	0.8027
Test support	929 epochs (Subject 015)		

TABLE XI: Overall test-set performance comparison. Best values in **bold**.

C. *Per-class Classification Reports*

Tables XII–XV show precision, recall, and F1-score per sleep stage for each model. Across all models, N1 is consistently the hardest class: recall ranges from 0.22 (EfficientNet-B0) to 0.60 (ResNet-18), reflecting severe class imbalance (only 58 N1 samples vs. 409 N2). N2, N3, and REM are well-classified by all models, with F1-scores above 0.82. The W class is reliably

recalled (≥ 0.89) but with moderate precision due to confusion with REM.

Class	Precision	Recall	F1	Support
W	0.7536	0.9123	0.8254	57
N1	0.4091	0.4655	0.4355	58
N2	0.9465	0.8655	0.9042	409
N3	0.9580	0.9514	0.9547	144
REM	0.7942	0.8429	0.8178	261
Macro avg	0.7723	0.8075	0.7875	929
Weighted avg	0.8601	0.8504	0.8537	929

TABLE XII: Per-class report: Custom CNN (Acc = 85.04%, κ = 0.7881)

Class	Precision	Recall	F1	Support
W	0.8226	0.8947	0.8571	57
N1	0.3804	0.6034	0.4667	58
N2	0.9721	0.8509	0.9074	409
N3	0.9329	0.9653	0.9488	144
REM	0.8545	0.8774	0.8658	261
Macro avg	0.7925	0.8383	0.8092	929
Weighted avg	0.8868	0.8633	0.8715	929

TABLE XIII: Per-class report: ResNet-18 (Acc = 86.33%, κ = 0.8086)

Class	Precision	Recall	F1	Support
W	0.6067	0.9474	0.7397	57
N1	0.3714	0.2241	0.2796	58
N2	0.9489	0.8631	0.9040	409
N3	0.9441	0.9375	0.9408	144
REM	0.7897	0.8774	0.8312	261
Macro avg	0.7322	0.7699	0.7390	929
Weighted avg	0.8464	0.8439	0.8402	929

TABLE XIV: Per-class report: EfficientNet-B0 (Acc = 84.39%, κ = 0.7783)

Class	Precision	Recall	F1	Support
W	0.8182	0.9474	0.8780	57
N1	0.4127	0.4483	0.4298	58
N2	0.9412	0.8606	0.8991	409
N3	0.9559	0.9028	0.9286	144
REM	0.8207	0.9119	0.8639	261
Macro avg	0.7897	0.8142	0.7999	929
Weighted avg	0.8691	0.8611	0.8632	929

TABLE XV: Per-class report: MobileNet V2 (Acc = 86.11%, κ = 0.8027)

D. *Confusion Matrices*

Tables XVI–XIX show the confusion matrices on the 929-sample test set. Rows represent true labels and columns represent predicted labels (W, N1, N2, N3, REM).

	W	N1	N2	N3	REM
W	**52**	1	3	1	0
N1	6	**27**	2	0	23
N2	4	12	**354**	5	34
N3	0	0	7	**137**	0
REM	7	26	8	0	**220**

TABLE XVI: Confusion matrix: Custom CNN

	W	N1	N2	N3	REM
W	**51**	3	0	2	1
N1	6	**35**	0	0	17
N2	2	31	**348**	7	21
N3	0	0	5	**139**	0
REM	3	23	5	1	**229**

TABLE XVII: Confusion matrix: ResNet-18

	W	N1	N2	N3	REM
W	**54**	2	0	0	1
N1	14	**13**	2	0	29
N2	9	8	**353**	8	31
N3	0	0	9	**135**	0
REM	12	12	8	0	**229**

TABLE XVIII: Confusion matrix: EfficientNet-B0

	W	N1	N2	N3	REM
W	**54**	2	0	1	0
N1	7	**26**	2	0	23
N2	4	19	**352**	5	29
N3	0	0	14	**130**	0
REM	1	16	6	0	**238**

TABLE XIX: Confusion matrix: MobileNet V2

The confusion matrices reveal a consistent cross-model pattern: N1 is frequently misclassified as REM (23 errors in Custom CNN, 17 in ResNet-18, 29 in EfficientNet-B0, 23 in MobileNet V2). This is understandable, as both N1 and REM are characterised by low-amplitude, mixed-frequency EEG activity, making their spectrograms visually similar. N2 is occasionally confused with N1 (10–31 errors), likely because K-complexes and sleep spindles in N2 can appear as low-power transients. N3 is the most reliably classified stage across all models, with at most 14 misclassifications, consistent with its distinctive high-amplitude delta activity.

VII. Conclusion

This paper presented a complete pipeline for automated sleep stage classification from single-channel EEG spectrograms using convolutional neural networks. Raw EEG signals from the DODH dataset were converted to time-frequency spectrogram images via the Short-Time Fourier Transform, and four CNN models: a Custom CNN trained from scratch, ResNet-18, EfficientNet-B0, and MobileNet V2 were trained and evaluated on a subject-independent held-out test set (Subject 015, 929 epochs).

ResNet-18 achieved the best overall performance with a test accuracy of 86.33% and a Cohen's Kappa of 0.8086,

indicating strong agreement with the ground-truth annotations. MobileNet V2 followed closely (86.11%, $\kappa = 0.8027$), while the Custom CNN and EfficientNet-B0 scored 85.04% and 84.39% respectively. All four models exceeded 84% accuracy and $\kappa > 0.77$ on the unseen subject, demonstrating that transfer learning from ImageNet provides a strong and reliable initialisation for spectrogram-based EEG classification.

Three key findings emerge from this study. First, transfer learning consistently outperforms training from scratch: even though the Custom CNN was specifically designed for this task, all three pretrained models matched or exceeded it with far fewer training epochs. Second, class imbalance remains the dominant challenge: stage N1 is the most sparsely represented class (58 samples, 6.2% of the test set) was the hardest to classify across all models, with macro-averaged recall as low as 0.22 (EfficientNet-B0). The rarity-weight strategy partially mitigated this, but N1's F1-score did not exceed 0.47 in any model. Third, the N1/REM confusion is a structural problem in spectral similarity: both stages exhibit low-amplitude mixed-frequency activity, making them visually indistinguishable without temporal context.

VIII. Future work

Future efforts may identify a more efficient method to improve models performance, some prediction like between **N1 & Rem** or **N2 & Rem** sometimes not correct, which may be not happen anymore due to the mode improvements. Furthermore, the data set is unbalanced, which cause bias prediction between classes, especially class **N2**, we consider to improve dataset quality to solve this problem.

IX. Acknowledgements

We would like to thank Prof. Tran Giang Son for his guidance throughout this project. We also thank the creators of the Sleep 30s Spectrograms from DODH Dataset for making their data publicly available. This project was developed using Google Colab with GPU support provided by Google

References

[1] E. O. Neftci, H. Mostafa, and F. Zenke, *snnTorch Tutorial 2: Training Spiking Neural Networks*, **Available**: https://snntorch.readthedocs.io/en/latest/tutorials/tutorial$_2.html$

[2] N. Elias, "Audio Classification of Low Feature Spectrograms Utilizing Convolutional Neural Networks," University of Texas at Austin, 2024. Available: https://arxiv.org/html/2410.21561v1

[3] Sleep Foundation, "Stages of Sleep," Available: https://www.sleepfoundation.org/stages-of-sleep

[4] K. He, X. Zhang, S. Ren, and J. Sun, *Deep Residual Learning for Image Recognition*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015.

[5] Mingxing Tan, Quoc Le. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks* Proceedings of the 36th International Conference on Machine Learning, PMLR 97:6105-6114, 2019.

[6] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, *MobileNetV2: Inverted Residuals and Linear Bottlenecks*, arXiv preprint arXiv:1801.04381, 2019.

[7] GeeksforGeeks, "Introduction to Convolutional Neural Network," Available: https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/

[8] McHugh M. L. (2012). Interrater reliability: the kappa statistic. Biochemia medica, 22(3), 276–282.

[9] TMSi Artinis, "Types of Brain Waves," Available: https://www.tmsi.artinis.com/blog/types-of-brain-waves